מבוא למדעי המחשב מ'/ח' (234114 \ 234117)

סמסטר אביב תשע"ט

מבחן מסכם מועד א', 25 ביולי 2019

מספר סטודנט: | | | | | | | | | |

רשום/ה לקורס: | | 1 | 1 | 4 | 3 | 2 |

משך המבחן: 3 שעות.

חומר עזר: אין להשתמש בכל חומר עזר.

הנחיות כלליות:

- מלאו את הפרטים בראש דף זה ובדף השער המצורף, בעט בלבד.
- בדקו שיש 22 עמודים (4 שאלות) במבחן, כולל עמוד זה.
- כתבו את התשובות על טופס המבחן בלבד, במקומות המיועדים לכך. שימו לב שהמקום המיועד לתשובה אינו מעיד בהכרח על אורך התשובה הנכונה.
- העמודים הזוגיים בבחינה ריקים. ניתן להשתמש בהם כדפי טיוטה וכן לכתיבת תשובותיכם. סמנו טיוטות באופן ברור על מנת שהן לא תיבדקנה.
- יש לכתוב באופן ברור, נקי ומסודר, **ובעט בלבד**.
- בכל השאלות, הנכם רשאים להגדיר ולממש פונקציות עזר כרצונכם. לנוחיותכם, אין חשיבות לסדר מימוש הפונקציות בשאלה, ובפרט ניתן לממש פונקציה לאחר השימוש בה.
- אלא אם כן נאמר אחרת בשאלות, **אין להשתמש בפונקציות ספריה או בפונקציות שמומשו בכיתה**, למעט פונקציות קלט/פלט והקצאת זיכרון (`malloc, free`). ניתן להשתמש בטיפוס `bool` המוגדר ב-`stdbool.h`
- אין להשתמש במשתנים סטטיים וגלובאליים אלא אם נדרשתם לכך מפורשות.
- כשאתם נדרשים לכתוב קוד באילוצי סיבוכיות זמן/מקום נתונים, אם לא תעמדו באילוצים אלה תוכלו לקבל בחזרה מקצת הנקודות אם תחשבו נכון ותציינו את הסיבוכיות שהצלחתם להשיג.
- נוהל "לא יודע": אם תכתבו בצורה ברורה "לא יודע/ת" על שאלה (או סעיף) שבה אתם נדרשים לקודד, תקבלו 20% מהניקוד. דבר זה מומלץ אם אתם יודעים שאתם לא יודעים את התשובה.
- נוסחאות שימושיות:

$$1+\frac{1}{2}+\frac{1}{3}+\ldots+\frac{1}{n}=\Theta(\log n) \qquad 1+\frac{1}{4}+\frac{1}{9}+\frac{1}{16}+\frac{1}{25}+\ldots=\Theta(1)$$

$$1^k+2^k+3^k+\ldots+n^k=\Theta(n^{k+1}) \qquad 1+k+k^2+k^3+\ldots+k^n=\Theta(k^n)$$

צוות הקורס 234114/7

מרצים: פרופ' מירלה בן-חן (מרצה אחראית), גב' יעל ארז **מתרגלים:** עמית ברכה, גיא ברשצקי, עמר דהרי, קטרין חדד, דניאל עזוז, דמיטרי רבינוביץ' (מתרגל אחראי)

בהצלחה!

שאלה 1 (25 נקודות):

א. (8 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה f1 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט שיקוליכם. חובה לפשט את הביטוי ככל שניתן. הניחו שסיבוכיות הזמן של malloc(n) ו-free היא Θ(1), וסיבוכיות המקום של malloc(n) היא Θ(n).

```
int f1(int n)
{
      int sum = 0;
      for (int i = 0; i < n; ++i)
      {
            int *b = malloc(sizeof(int) * i);

            for (int j = 1; j < i; j *= 2)
                  b[j] = i * i;

            sum += b[i / 2];
            free(b);
      }
      return sum;
}
```

סיבוכיות זמן: Θ(_____) סיבוכיות מקום: Θ(_____)

ב. (9 נקודות): חשבו את סיבוכיות הזמן והמקום של הפונקציה f2 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט שיקוליכם. חובה לפשט את הביטוי ככל שניתן. הניחו שסיבוכיות הזמן של malloc(n) ו-free היא Θ(1), וסיבוכיות המקום של malloc(n) היא Θ(n).

```
int f2(int n)
{
      if (n <= 1) return 1;
      int *arr = malloc(sizeof(int) * n);

      for (int i = 0; i < n; ++i)
            arr[i] = i;

      free(arr);
      return 2 * f2(n / 2) + 1;
}
```

סיבוכיות זמן: Θ(_____) סיבוכיות מקום: Θ(_____)

ג. (8 נקודות): חשבו את סיבוכיות הזמן והמקום של הפונקציה f3 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט שיקוליכם. חובה לפשט את הביטוי ככל שניתן.

```
int g(int n)
{
      if (n <= 1) return 1;
      return g(n / 2) + 1;
}

int f3(int n)
{
      int counter = 0;

      for (int i = 0; g(i) < n; ++i)
            ++counter;

      return counter;
}
```

סיבוכיות זמן: Θ(_____) סיבוכיות מקום: Θ(_____)

שאלה 2 (25 נק')

נתון מערך `arr`, לא בהכרח ממוין שמכיל מספרים שלמים. כל המספרים במערך מופיעים בזוגות עוקבים, פרט למספר אחד בדיוק, שלו יש מופע בודד חסר בן-זוג עוקב.

- לא ייתכנו יותר משני מספרים זהים עוקבים.
- זוג מספרים זהים יכול להופיע יותר מפעם אחת, אבל לא ברצף.
- המספר מחוסר בן הזוג יכול להופיע גם כזוג במקום אחר במערך, אבל לא ברצף.

ממשו פונקציה

```
int FindOddOccuring(int arr[], int n)
```

שתקבל את המערך `arr` ואורכו `n` ותחזיר את האיבר שיש לו הופעה בודדת.

לדוגמה: אם נתון מערך הבא

8	8	5	5	3	6	6	−1	−1	7	7

הפונקציה תחזיר 3.

ועבור מערך

1	1	5	5	1	1	5	1	1

הפונקציה תחזיר 5.

דרישות:

- ממשו את הפונקציה בצורה היעילה ביותר .

אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

`int FindOddOccuring(int arr[], int n)`
{

שאלה 3 (25 נקודות) :

"טריפת אותיות", או "אנגרמה" בלעז, היא יצירת מילה חדשה מערבוב אותיות של מילה קיימת, או משפט חדש מערבוב אותיות של משפט קיים. לדוגמא, "Tom Marvolo Riddle" היא אנגרמה של המילה "I am Lord Voldemort". **שימו לב** כי אין משמעות לרווחים במשפט וכי לא מבדילים בין אותיות קטנות לגדולות.

ממשו את הפונקציה AreAnagrams המקבלת שתי מחרוזות s1 ו- s2 ומחזירה true אם הן אנגרמות אחת של השנייה, או false אחרת:

```
bool AreAnagrams(char * s1, char * s2)
```

שימו לב: ניתן להניח כי s1 ו- s2 מורכבות מאותיות באנגלית ורווחים בלבד.

דרישות:

- סיבוכיות זמן $O(n+m)$, כאשר n אורך המחרוזת s1 ו- m אורך המחרוזת s2, וסיבוכיות מקום נוסף $O(1)$.

אם לפי חישוביכם לא עמדתם בדרישות הסיבוכיות אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

```
bool AreAnagrams(char * s1, char * s2){
```

שאלה 4 (25 נקודות) :

בהינתן מערך חד ממדי `letters` עם אותיות באנגלית, ומערך דו ממדי `matrix` עם אותיות באנגלית, אנו נתעניין במציאת האורך של המסלול הארוך ביותר ב – `matrix` שמורכב כולו מאותיות מהמערך `letters`. המסלול צריך להיות רציף, פשוט (אינו חוצה את עצמו) ולכל שני תאים עוקבים במסלול צריכה להיות צלע משותפת.

```
int FindLongestPath(char letters[N_ALPH], char matrix[N][M])
```

דוגמה:

בהינתן מערך `letters={a, b}` ומטריצה `matrix`

b	b	c	f	e	a	a
b	c	a	a	b	f	b
f	e	a	e	b	c	b
e	f	b	b	a	d	b
d	a	a	a	e	c	a
c	d	e	f	f	a	b

הפונקציה תחזיר `11` (ראו מסלול במרכז).

הערות:

- יש להשתמש בשיטת `backtracking` כפי שנלמדה בכיתה.
- בשאלה זו אין דרישות סיבוכיות, אולם כמקובל ב-`backtracking` יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר (ויש לממש את כולן).
- `N`, `M`, `N_ALPH` הם קבועים המוגדרים ב-`define`

```
int FindLongestPath(char letters[N_ALPH], char matrix[N][M])
{
```